

Answers to the exercises about regular expressions

Christian Rinderknecht

4 October 2005

Question 1. Identify the lexemes and their corresponding token in the following C program.

```
/* Return maximum of integers i and j */  
int max (int i, int j) {  
    return i>j? i : j;  
}
```

Answer 1. The token may be different for different languages. Even if we do not know exactly their names for the C language, we still can find meaningful names based on what they denote. Also we may decide or not to make a token for each keyword. This is a creative work! Therefore several solutions may be acceptable as long as they enable a meaningful implementation. However, it is usual to call the identifiers “identifiers”. One answer to the question is therefore

TOKEN	LEXEME	TOKEN	LEXEME
keyword	int	keyword	return
identifier	max	identifier	i
symbol	(	relation	>
keyword	int	identifier	j
identifier	i	symbol	?
symbol	,	identifier	i
keyword	int	symbol	:
identifier	j	identifier	j
symbol	)	terminator	;
symbol	{	symbol	}

Note that the comments are recognised by the lexer and then discarded, i.e., they are not transmitted to the parser. The same happens to spaces, i.e., blanks and tabulations, which are used for separating lexemes in the source but are not meaningful by themselves.

Question 2. Let the alphabet $\Sigma = \{a, b\}$ and the following regular expressions:

$$r = a(a|b)^*ba$$

$$s = (ab)^*|(ba)^*|(a^*|b^*)$$

The language denoted by r is noted $L(r)$ and the language denoted by s is noted $L(s)$.

Find a word x such as

1. $x \in L(r)$ and $x \notin L(s)$
2. $x \notin L(r)$ and $x \in L(s)$
3. $x \in L(r)$ and $x \in L(s)$
4. $x \notin L(r)$ and $x \notin L(s)$

Answer 2. The method to answer these questions is simply to try small words by constructing them in order to satisfy the constraints.

1. The shortest word x belonging to $L(r)$ is found by taking ε in place of $(a|b)^*$. So $x = aba$. Let us check if $x \in L(s)$ or not. $L(s)$ is made of the union of four sub-languages (subsets). To make this clear, let us remove the useless parentheses on the right side:

$$s = (ab)^* | (ba)^* | a^* | b^*$$

Therefore, membership tests on $L(s)$ have to be split into four: one membership test on $(ab)^*$, one on $(ba)^*$, one on a^* and another one on b^* . In other words:

$$x \in L(s) \Leftrightarrow x \in L((ab)^*) \text{ or } x \in L((ba)^*) \text{ or } x \in L(a^*) \text{ or } x \in L(b^*)$$

Let us test the membership with $x = aba$:

- (a) The words in $L((ab)^*)$ are $\varepsilon, ab, abab \dots$. Thus $aba \notin L((ab)^*)$.
- (b) The words in $L((ba)^*)$ are $\varepsilon, ba, baba \dots$. Hence $aba \notin L((ba)^*)$.
- (c) The words in $L(a^*)$ are $\varepsilon, a, aa \dots$. Therefore $aba \notin L(a^*)$.
- (d) The words in $L(b^*)$ are $\varepsilon, b, bb \dots$. So $aba \notin L(b^*)$.

Finally the conclusion is $aba \notin L(s)$, which is what we were looking for.

2. What is the shortest word belonging to $L(s)$? Since the four sub-languages composing $L(s)$ are starred, it means that $\varepsilon \in L(s)$. Since we showed at the item (1) that aba is the shortest word of $L(r)$, it means that $\varepsilon \notin L(r)$ because ε is of length 0.
3. This question is a bit more difficult. After a few tries, we cannot find any x such as $x \in L(r)$ and $x \in L(s)$. Then we may try to prove that $L(r) \cap L(s) = \emptyset$,

i.e., there is no such x . How should we proceed? The idea is to use the decomposition of $L(s)$ into four sub-languages and try to prove

$$L(r) \cap L((ab)^*) = \emptyset$$

$$L(r) \cap L((ba)^*) = \emptyset$$

$$L(r) \cap L(a^*) = \emptyset$$

$$L(r) \cap L(b^*) = \emptyset$$

Indeed, if all these four equations are true, they imply $L(r) \cap L(s) = \emptyset$.

- (a) Any word in $L(r)$ ends with a whereas any word in $L((ab)^*)$ finishes with b or is ε . Thus $L(r) \cap L((ab)^*) = \emptyset$.
- (b) For the same reason, $L(r) \cap L(b^*) = \emptyset$.
- (c) Any word in $L(r)$ contains both a and b whereas any word in $L(a^*)$ contains only a or is ε . Therefore $L(r) \cap L(a^*) = \emptyset$.
- (d) Any word in $L(r)$ starts with a whereas any word in $L((ba)^*)$ starts with b or is ε . Thus $L(r) \cap L((ba)^*) = \emptyset$.

Finally, since all the four equations are false, they imply that

$$L(r) \cap L(s) = \emptyset$$

4. Let us construct letter by letter a word x which does not belong neither to $L(r)$ nor $L(s)$. First, we note that all words in $L(r)$ start with a , so we can try to start x with b : this way $x \notin L(r)$. So we have $x = b \dots$ and we have to fill the dots with some letters in such a way that $x \notin L(s)$.

We use again the decomposition of $L(s)$ into four sub-languages and make sure that x does not belong to any of those sub-languages.

First, because x starts with b , $x \notin L(a^*)$ and $x \notin L((ab)^*)$.

Now, we have to add some more letters such that $x \notin L(b^*)$ and $x \notin L((ba)^*)$.

Since any word in $L(b^*)$ has a letter b as second letter or is ε , we can choose the second letter of x to be a . This way $x = ba \dots \notin L(b^*)$.

Finally, we have to add more letters to make sure that

$$x = ba \dots \notin L((ba)^*)$$

Any word in $L((ba)^*)$ is either ε or ba or $baba \dots$, hence the third letter is b . Therefore, let us choose the letter a as the third letter of x and we thus have $x = baa \notin L((ba)^*)$. Summary:

$$baa \notin L(r), baa \notin L(b^*), baa \notin L((ba)^*), baa \notin L(a^*), baa \notin L((ab)^*)$$

which is equivalent to

$$baa \notin L(r) \text{ and } baa \notin L((ab)^*) \cup L((ba)^*) \cup L(a^*) \cup L(b^*) = L(s)$$

Therefore $x = baa$ is one possible answer.

Question 3. Given the binary alphabet $\Sigma = \{a, b\}$ and the order on letters $a < b$, write regular definitions for the following languages.

1. All words starting and ending with a .
2. All non-empty words.
3. All words in which the third last letter is a .
4. All words containing exactly three a .
5. All words containing at least one a before a b .
6. All words in which the letters are in increasing order.
7. All words with no letter following the same one.

Answer 3. When answering these questions, it is important to keep in mind that the language of words made up on the alphabet Σ is Σ^* and that there are, in general, several regular expressions describing one language.

1. The constraint on the words is that they must be of the shape $a \dots a$ where the dots stand for “any combination of a and b .” In other words, one answer is $a(a|b)^*a|a$.
2. This question is very simple since the language of all words is $(a|b)^*$, we have to remove ε , i.e., one simple answer is $(a|b)^+$.
3. The question implies that the words we are looking for are of the form $\dots a _ _$ where the dots stand for “any sequence of a and b ” and each $_$ stands for a regular expression denoting any letter. Any letter is described by $(a|b)$; therefore one possible answer is $(a|b)^*a(a|b)(a|b)$.
4. The words we search contain, at any place, exactly three a , so are of the form $\dots a \dots a \dots a \dots$, where the dots stand for “any letter except a ”, i.e., “any number of b .” In other words: $b^*ab^*ab^*ab^*$.
5. Because the alphabet contains only two letters, the question is equivalent to: “All words containing the substring ab ”, i.e., the words are of the form $\dots ab \dots$ where the dots stand for “any sequence of a and b .” It is then easy to understand that a short answer is $(a|b)^*ab(a|b)^*$.
6. Because the alphabet is made only of two letters, the answer is easy: we put first all the a and then all the b : a^*b^* .
7. Since the alphabet contains only two letters, the only way to not repeat a letter is to only have substrings ab or ba in the words we look for. In other words: $abab \dots ab$ or $abab \dots aba$ or $baba \dots ba$ or $baba \dots bab$. In short: $(ab)^*a?(ba)^*b?$ or, even shorter: $a?(ba)^*b?$.

Question 3bis. Let the alphabet be now the ASCII. Find regular definitions for

1. All comments on one line between braces, i.e., { and }.
2. All comments between /* and */ without */ inside, except between double-quotes, i.e ".

Hint: Use the complement of a letter in the alphabet, e.g., $\bar{A}$ denotes all letters except A. g

Answer 3bis.

1. Here, the key point is how to *exclude* some letters or combinations of letters from the comment. We must use regular expressions corresponding to the *complement* operation on languages of one letter in the alphabet, e.g. $\bar{a}$ denotes all the letters in the alphabet except a . Therefore, since we have to exclude the end-of-line character and the `}` symbol inside the comments, the regular expression for any letter different from `}` and `\n` is $(\bar{\text{}}|\backslash n)$. Therefore, we can write the following regular definitions:

$$\begin{aligned} \text{inside} &\rightarrow \overline{(\}|\backslash n) \\ \text{comment} &\rightarrow \{\text{inside}^*\} \end{aligned}$$

2. Here, the delimiters of the comments are `/*` and `*/`, so we can start writing

```
inside →
comment → /* inside*/
```

We have three cases in **inside**:

- (a) it is a string between double-quotes,
- (b) it is * not followed by /,
- (c) it is a letter different from * or "

These cases are described by the following regular expressions:

- (a) **string**
(b) $\ast \overline{}$
(c) $\overline{\ast | }$

In other words:

```
string →
inside → string | */ | *| "
comment → /* inside**/
```

The string cannot contain a double-quote, otherwise there is no way to determine where the string finishes.

$\text{string} \rightarrow "(\overline{) }^*"$
 $\text{inside} \rightarrow \text{string} | \overline{ * / } | \overline{ * | }"$
 $\text{comment} \rightarrow /* \text{inside}^* */$

Question 4. Simplify, if possible, the following regular expressions.

$$\begin{aligned}
 &(\varepsilon | a^* | b^* | a | b)^* \\
 &a(a | b)^* b | (ab)^* | (ba)^*
 \end{aligned}$$

Answer 4.

1. The first regular expression can be simplified in the following way:

$$\begin{aligned}
 (\varepsilon | a^* | b^* | a | b)^* &= (\varepsilon | a^* | b^* | b)^* && \text{since } L(a) \subset L(a^*) \\
 &= (\varepsilon | a^* | b^*)^* && \text{since } L(b) \subset L(b^*) \\
 &= (\varepsilon | a^+ | b^+)^* && \text{since } \{\varepsilon\} \subset L(x^*) \\
 &= (a^+ | b^+)^* && \text{since } (\varepsilon | x)^* = x^*
 \end{aligned}$$

Words in $L((a^+ | b^+)^*)$ are of the form ε or $(a \dots a)(b \dots b)(a \dots a)(b \dots b) \dots$ where the dots stand for "repetition any number of time, including zero." So we recognise $(a | b)^*$. Therefore $(\varepsilon | a^* | b^* | a | b)^* = (a | b)^*$.

2. The second regular expression can be simplified in the following way. We note first that the expression is made of the disjunction of three regular sub-expressions (i.e., it is a union of three sub-languages). The simplest idea is then to check whether one of these sub-languages is redundant, i.e., if one is included in another. If so, we can simply remove it from the expression.

$$\begin{aligned}
 a(a | b)^* b | (ab)^* | (ba)^* &= a(a | b)^* b | \varepsilon | (ab)^+ | (ba)^* && \text{since } (ab)^* = \varepsilon | (ab)^+ \\
 &= a(a | b)^* b | (ab)^+ | (ba)^* && \text{since } \{\varepsilon\} \subset L((ba)^*)
 \end{aligned}$$

We have:

$$\begin{aligned}
 (ab)^+ &= (ab)(ab) \dots (ab) \\
 &= a(ba)(ba) \dots (ba)b | ab \\
 &= a(ba)^* b
 \end{aligned}$$

Also $L((ba)) \subset L((a | b)^*)$ and then $L((ba)^*) \subset L((a | b)^*)$, because $(a | b)^*$ denotes all the words. Therefore

$$\begin{aligned}
 L(a(ba)^* b) &\subset L(a(a | b)^* b) \\
 L((ab)^+) &\subset L(a(a | b)^* b)
 \end{aligned}$$

As a consequence, one possible answer is

$$a(a | b)^* b | (ab)^* | (ba)^* = a(a | b)^* b | (ba)^*$$

The intersection between $L(a(a|b)^*b)$ and $L((ba)^*)$ is empty because all the words of the former start with a , while all the words of the other start with b (or is ε). Therefore we cannot simply further this way.